

Имитационное моделирование

Лабораторная работа № 6. SIR-модель через сети Петри

Исаева Зарина Исмайлбековна

2026-09-07

Информация

Докладчик

- Исаева Зарина Исмайлбековна
- студенческий билет: **1132232882**
- дисциплина: «Имитационное моделирование»
- лабораторная работа № 6

Тема и цель

Тема: модель распространения эпидемии SIR в форме сети Петри.

Цель: связать структурное описание переходов, детерминированное решение и событийную стохастическую симуляцию в одном воспроизводимом проекте.

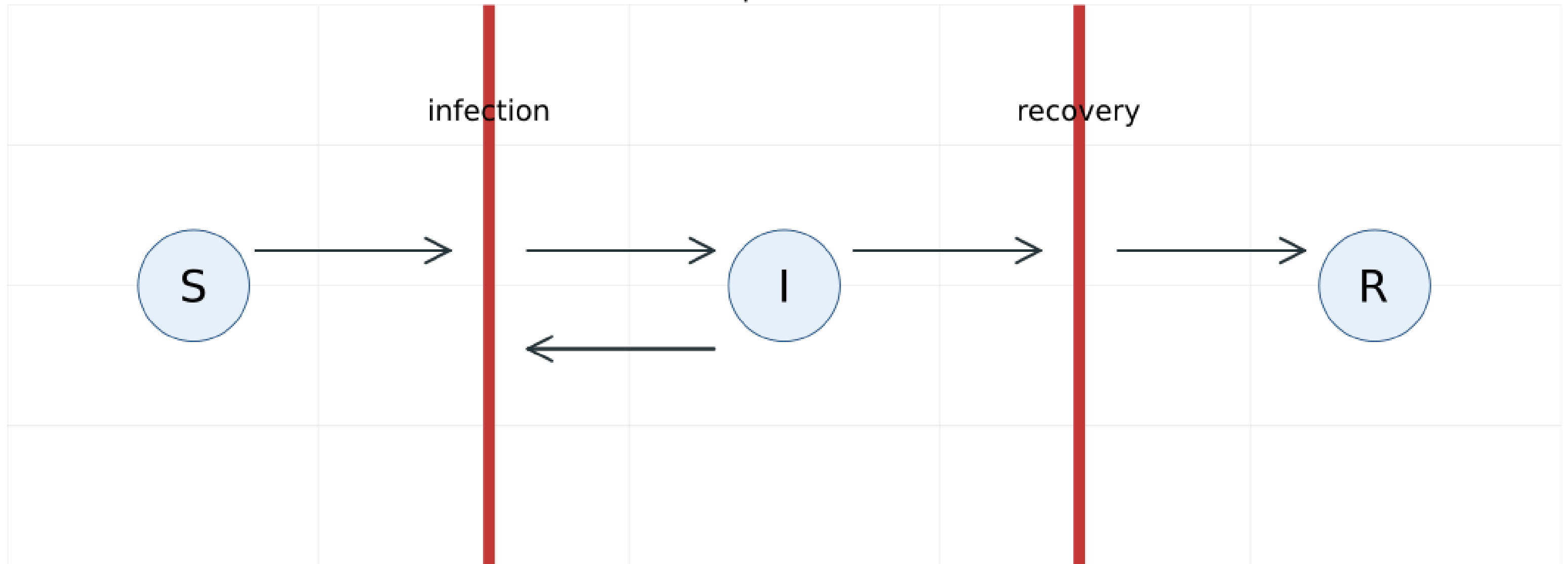
Задачи

1. Построить `LabelledPetriNet` с местами S, I, R.
2. Получить ODE- и SSA-траектории.
3. Просканировать 15 значений β .
4. Построить анимацию из 501 состояния.
5. Исследовать дополнительные параметры и проверить результаты.

Модель

Переходы сети Петри

Сеть Петри модели SIR



Сеть Петри SIR

Закон массового действия



$$\dot{S} = -\beta SI, \quad \dot{I} = \beta SI - \gamma I, \quad \dot{R} = \gamma I$$

В лабораторной работе используется именно βSI **без деления на N**.

Исходные параметры

Параметр	Значение
S_0, I_0, R_0	990, 10, 0
β, γ	0.3, 0.1
Горизонт	100
Шаг сохранения ODE	0.5
Seed SSA	123
Шаг анимации	0.2

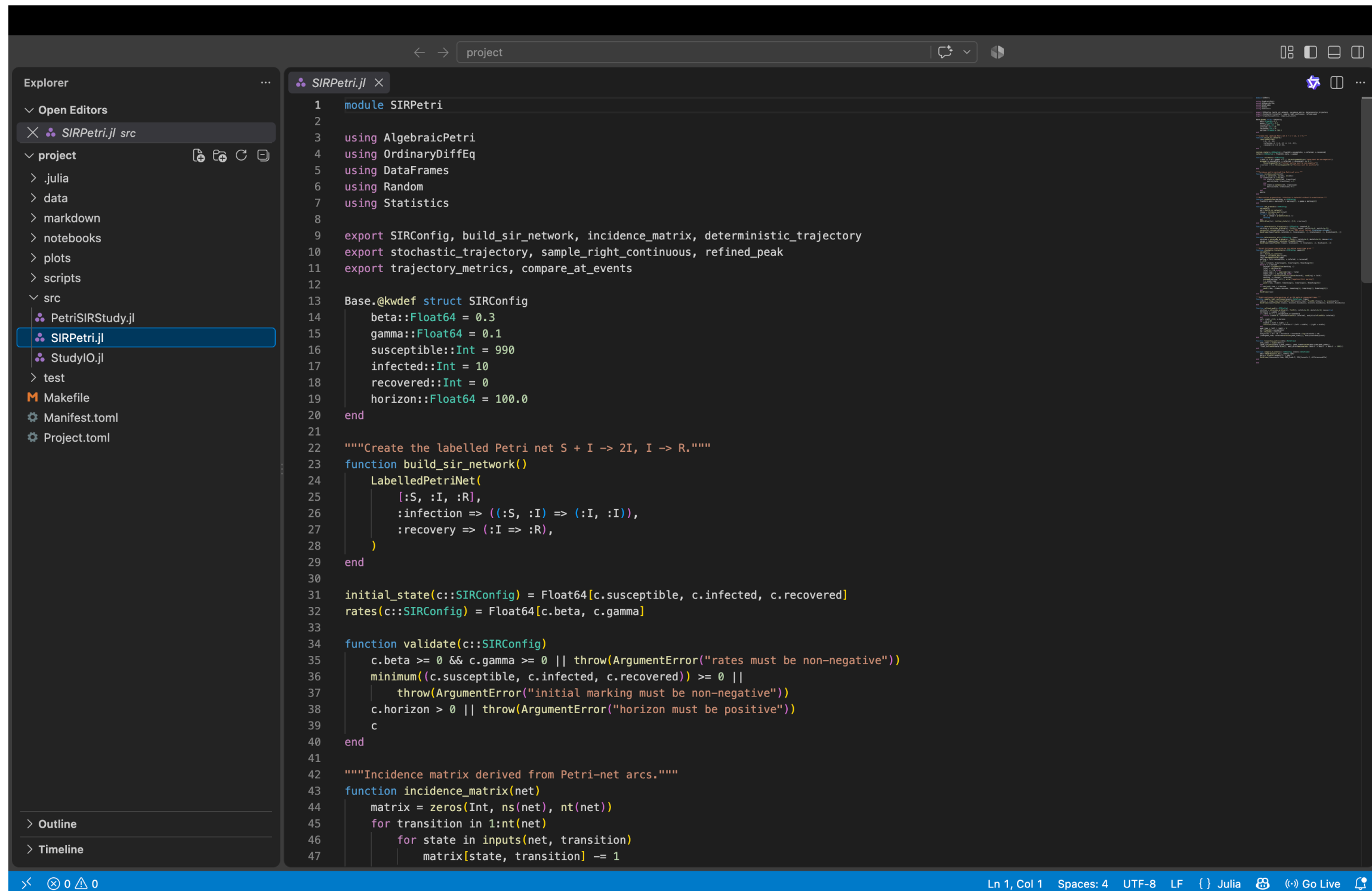
Матрица инцидентности

Место	infection	recovery
S	-1	0
I	+1	-1
R	0	+1

Столбцы суммируются в ноль, поэтому $S + I + R = 1000$.

Реализация

Архитектура проекта



The screenshot displays the Julia IDE interface. On the left, the Explorer pane shows the project structure:

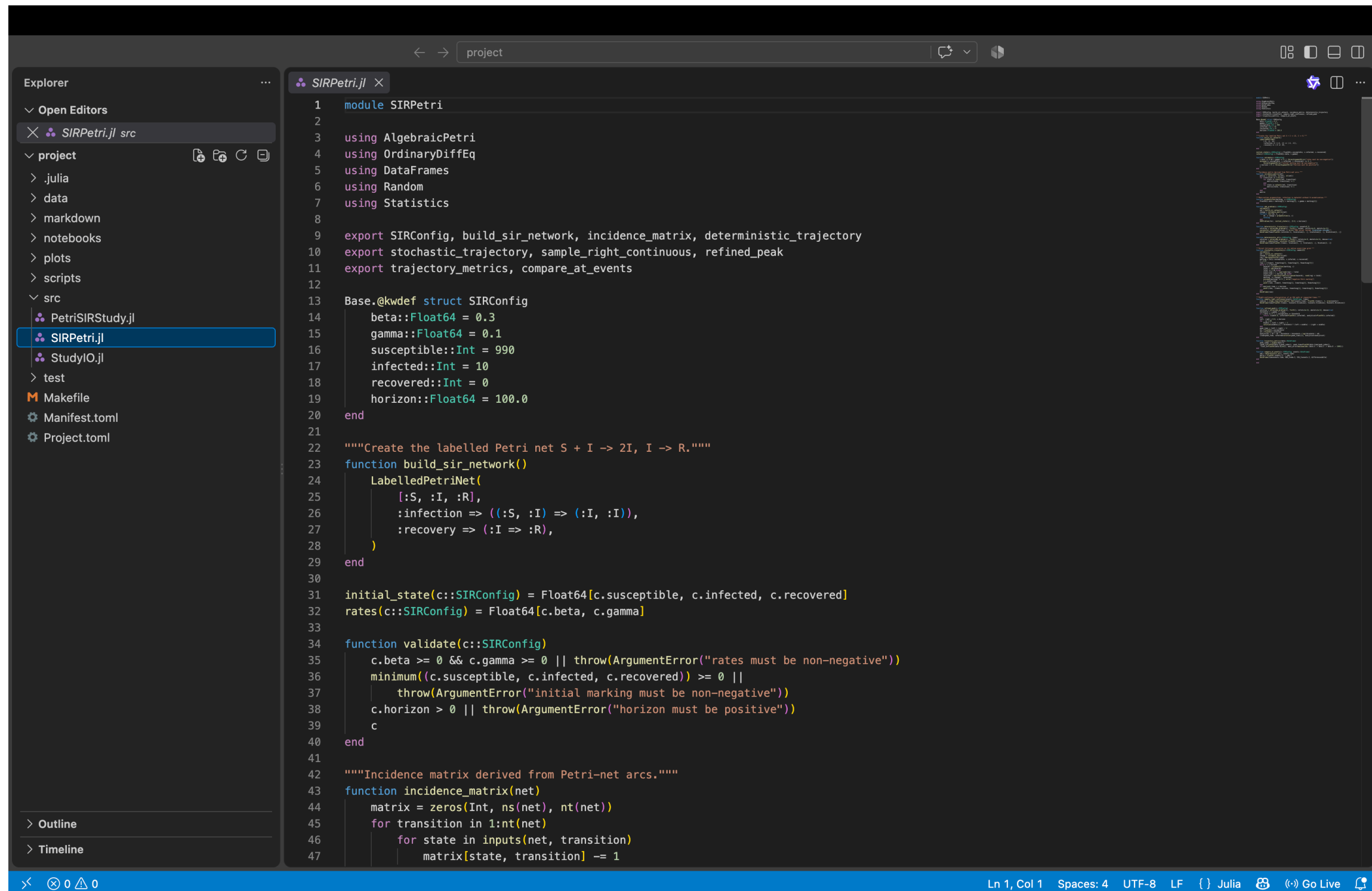
- Open Editors
 - SIRPetri.jl src
- project
 - .julia
 - data
 - markdown
 - notebooks
 - plots
 - scripts
 - src
 - PetriSIRStudy.jl
 - SIRPetri.jl**
 - StudyIO.jl
 - test
 - Makefile
 - Manifest.toml
 - Project.toml
- Outline
- Timeline

The main editor shows the content of `SIRPetri.jl`:

```
1 module SIRPetri
2
3 using AlgebraicPetri
4 using OrdinaryDiffEq
5 using DataFrames
6 using Random
7 using Statistics
8
9 export SIRConfig, build_sir_network, incidence_matrix, deterministic_trajectory
10 export stochastic_trajectory, sample_right_continuous, refined_peak
11 export trajectory_metrics, compare_at_events
12
13 Base.@kwdef struct SIRConfig
14     beta::Float64 = 0.3
15     gamma::Float64 = 0.1
16     susceptible::Int = 990
17     infected::Int = 10
18     recovered::Int = 0
19     horizon::Float64 = 100.0
20 end
21
22 """Create the labelled Petri net S + I -> 2I, I -> R."""
23 function build_sir_network()
24     LabelledPetriNet(
25         [:S, :I, :R],
26         :infection => (:S, :I) => (:I, :I),
27         :recovery => (:I => :R),
28     )
29 end
30
31 initial_state(c::SIRConfig) = Float64[c.susceptible, c.infected, c.recovered]
32 rates(c::SIRConfig) = Float64[c.beta, c.gamma]
33
34 function validate(c::SIRConfig)
35     c.beta >= 0 && c.gamma >= 0 || throw(ArgumentError("rates must be non-negative"))
36     minimum((c.susceptible, c.infected, c.recovered)) >= 0 ||
37         throw(ArgumentError("initial marking must be non-negative"))
38     c.horizon > 0 || throw(ArgumentError("horizon must be positive"))
39     c
40 end
41
42 """Incidence matrix derived from Petri-net arcs."""
43 function incidence_matrix(net)
44     matrix = zeros{Int, ns(net), nt(net)}
45     for transition in 1:nt(net)
46         for state in inputs(net, transition)
47             matrix[state, transition] -= 1
```

The status bar at the bottom indicates: Ln 1, Col 1 | Spaces: 4 | UTF-8 | LF | Julia | Go Live.

Исходный модуль в VS Code



Выполненный ноутбук в JupyterLab

The screenshot displays a JupyterLab interface with a notebook titled "sirpetri_run.ipynb". The notebook is in "read-only" mode and shows a Julia script for a Petri net-based SIR model simulation. The script includes the following code:

```
[1]: ENV["GKSwstype"] = "100"
using CSV, DataFrames, Plots, Statistics
include(joinpath(dirname(Base.active_project()), "src", "SIRPetri.jl"))
include(joinpath(dirname(Base.active_project()), "src", "StudyIO.jl"))
using .SIRPetri, .StudyIO

group = "sirpetri-run"
config = SIRConfig()
network = build_sir_network()
change = incidence_matrix(network)

deterministic = deterministic_trajectory(config; saveat=0.5)
stochastic = stochastic_trajectory(config; seed=123)
event_comparison = compare_at_events(config, stochastic)
peak = refined_peak(config)

save_csv(group, "sir_deterministic.csv", deterministic)
save_csv(group, "sir_stochastic_events.csv", stochastic)
save_csv(group, "event_time_comparison.csv", event_comparison)
save_csv(group, "refined_peak.csv", DataFrame(
    peak_time=[peak.time], numerical_peak=[peak.infected], analytical_peak=[peak.analytical],
    absolute_error=[abs(peak.infected - peak.analytical)]))
save_csv(group, "incidence_matrix.csv", DataFrame(state=["S", "I", "R"],
    infection=change[:, 1], recovery=change[:, 2]))

save_plot(group, "petri-network.png", petri_diagram())
save_plot(group, "deterministic-dynamics.png",
    sir_plot(deterministic; title="Детерминированная модель: β=0.3, γ=0.1"))
save_plot(group, "stochastic-dynamics.png",
    sir_plot(stochastic; title="Стохастическая модель: seed=123", stepped=true))

metrics = DataFrame(method=["ODE", "SSA"],
    peak_I=[peak.infected, maximum(stochastic.I)],
    peak_time=[peak.time, stochastic.time[argmax(stochastic.I)]],
    final_R=[deterministic.R[end], stochastic.R[end]],
    population_error=[maximum(abs.(deterministic.S + deterministic.I + deterministic.R .- 1000)),
        maximum(abs.(stochastic.S + stochastic.I + stochastic.R .- 1000))])
save_csv(group, "summary.csv", metrics)

println("Petri places/transitions: $(length(change[:, 1]))/$(length(change[1, :]))")
```


Объект сети

```
1 function build_sir_network()  
2   LabelledPetriNet(  
3     [:S, :I, :R],  
4     :infection => ([:S, :I] => [:I, :I]),  
5     :recovery => (:I => :R),  
6   )  
7 end
```

Детерминированная модель

```
1 function rhs!(du, u, _, _)  
2     du .= change * propensities(u, config)  
3     nothing  
4 end  
5 solution = solve(ODEProblem(rhs!, u0, (0.0, 100.0)), Tsit5())
```


Прямой алгоритм Гиллеспи

```
1 hazards = [beta*S*I, gamma*I]
2 event_time = t - log(rand(rng)) / sum(hazards)
3 selected = searchsortedfirst(cumsum(hazards), rand(rng)*sum(hazards))
4 marking .+= incidence[:, selected]
```

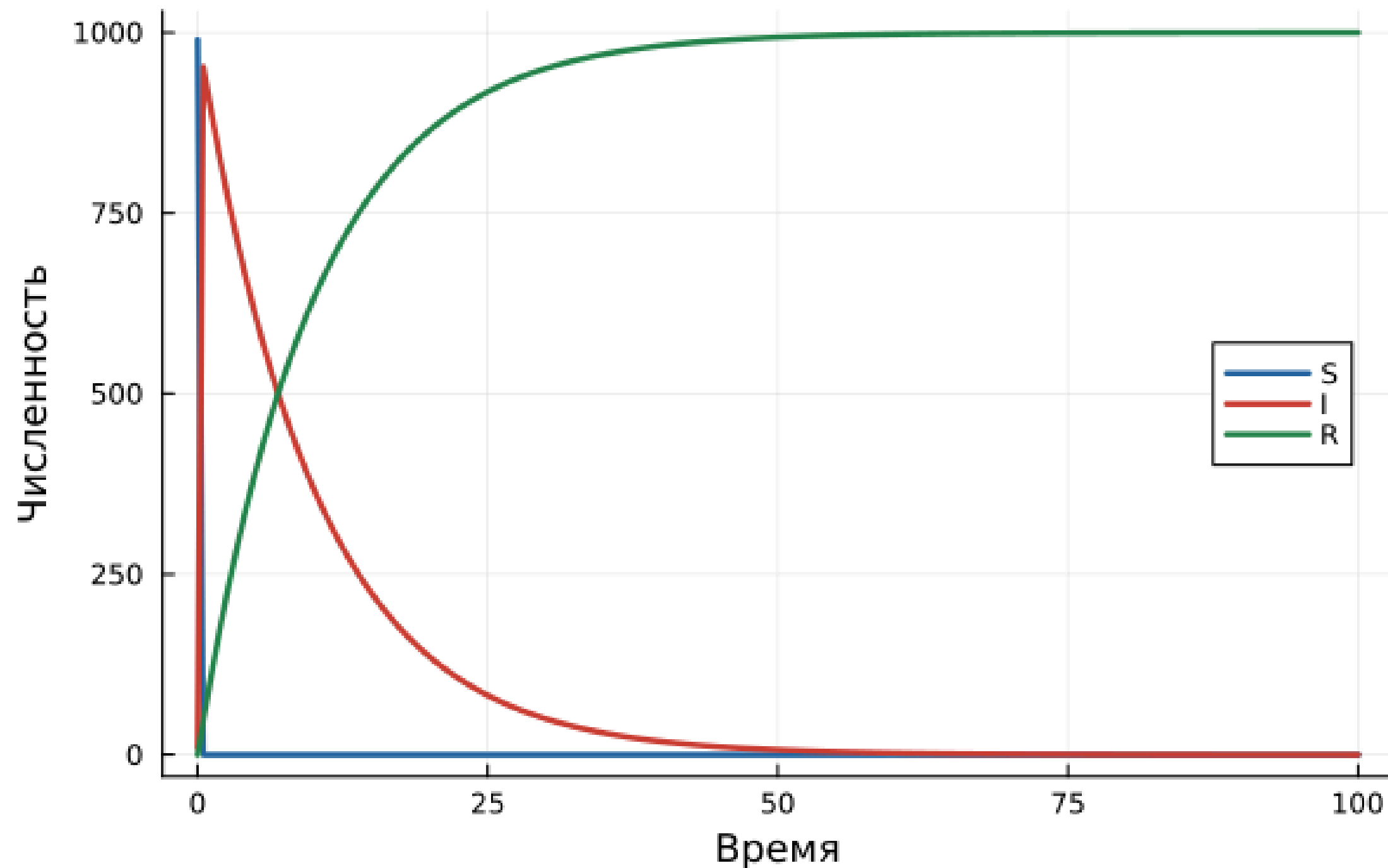
Литературные сценарии

- четыре обязательных файла `sirpetri_*.jl`;
- четыре самостоятельных параметрических варианта;
- для каждого: чистый `.jl`, выполненный `.ipynb`, `.qmd` и `.html`;
- отчётные фрагменты содержат полный листинг.

Основные результаты

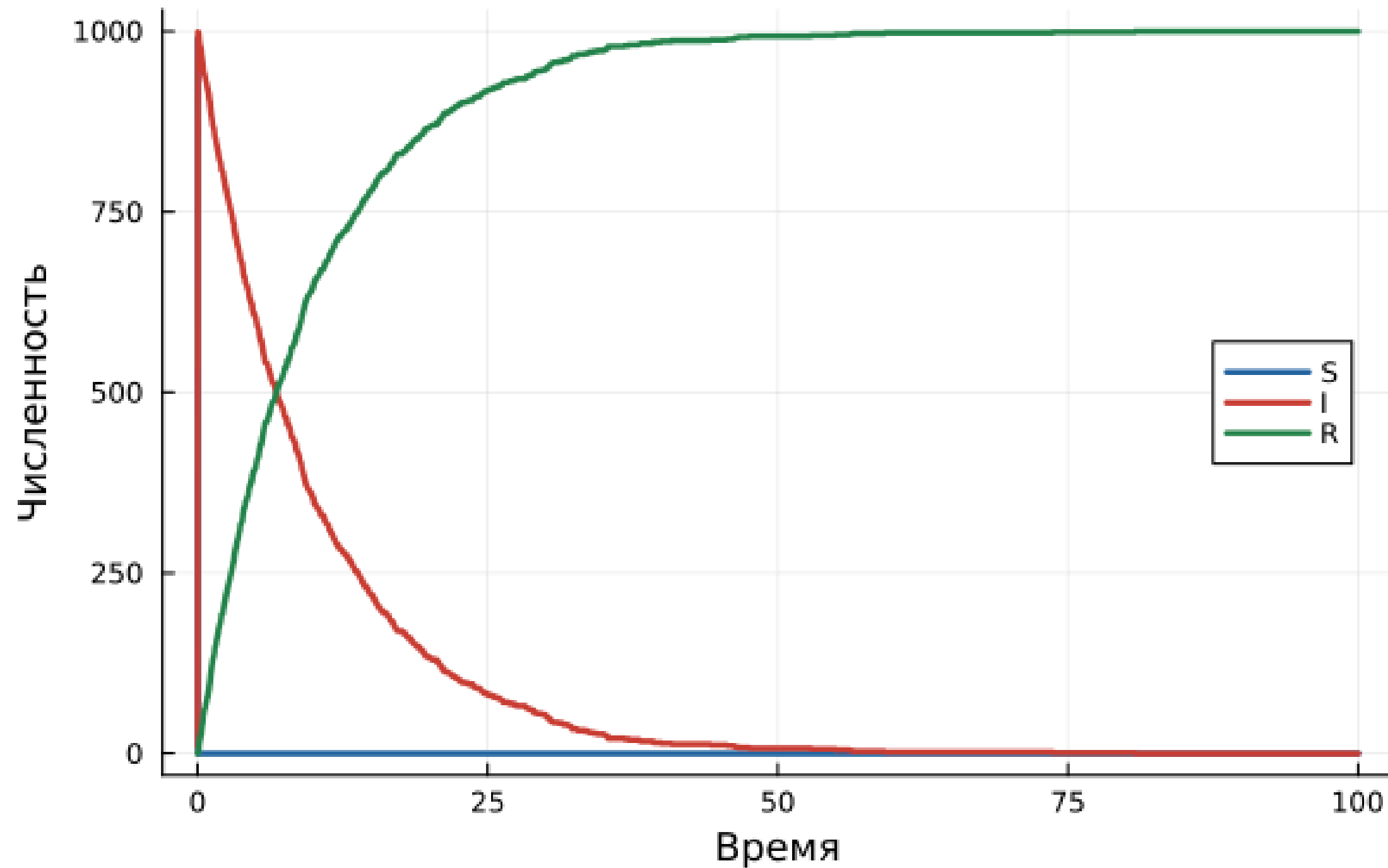
Детерминированная траектория

Детерминированная модель: $\beta=0.3$, $\gamma=0.1$

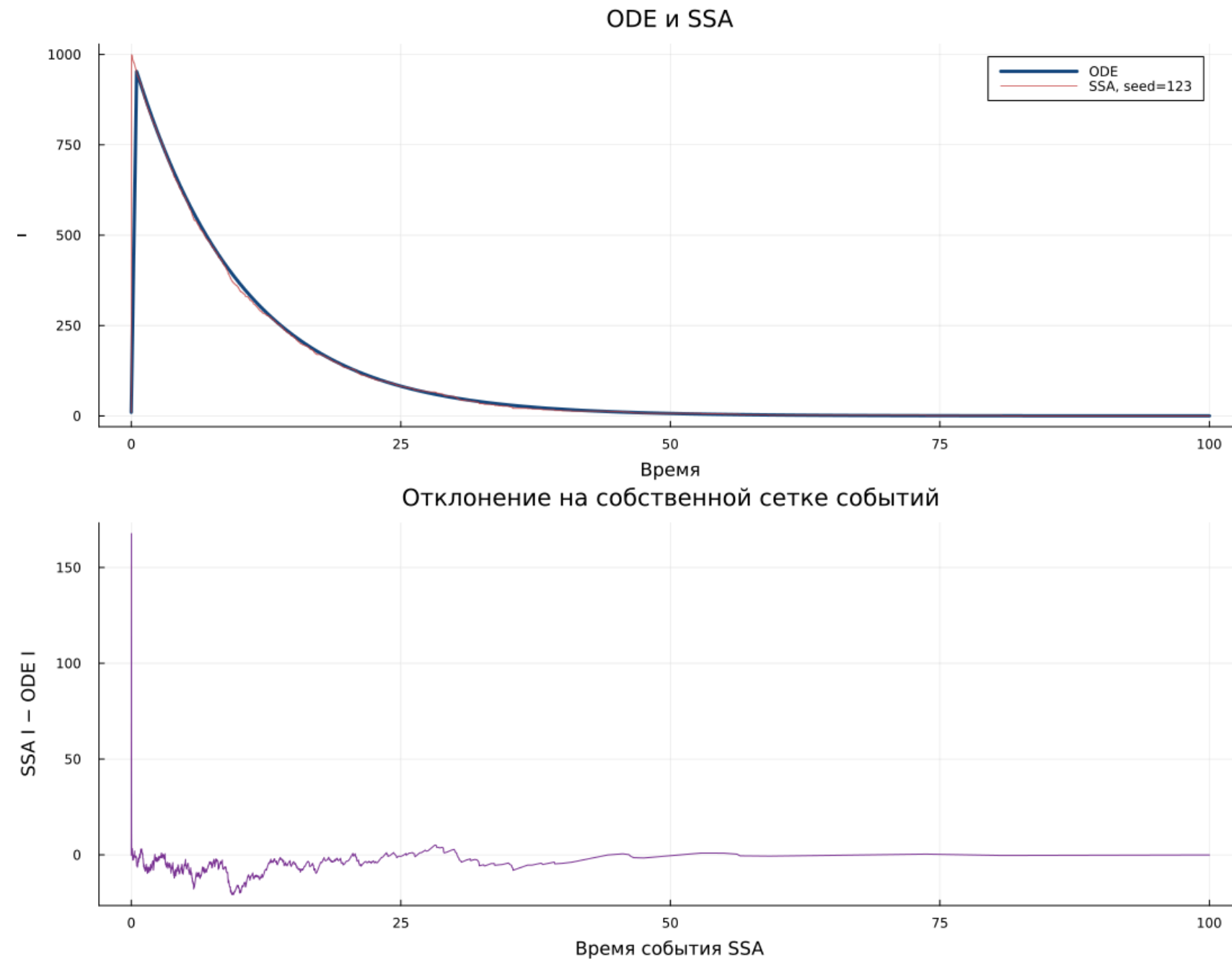


Стохастическая траектория

Стохастическая модель: seed=123



Корректное сравнение ODE и SSA



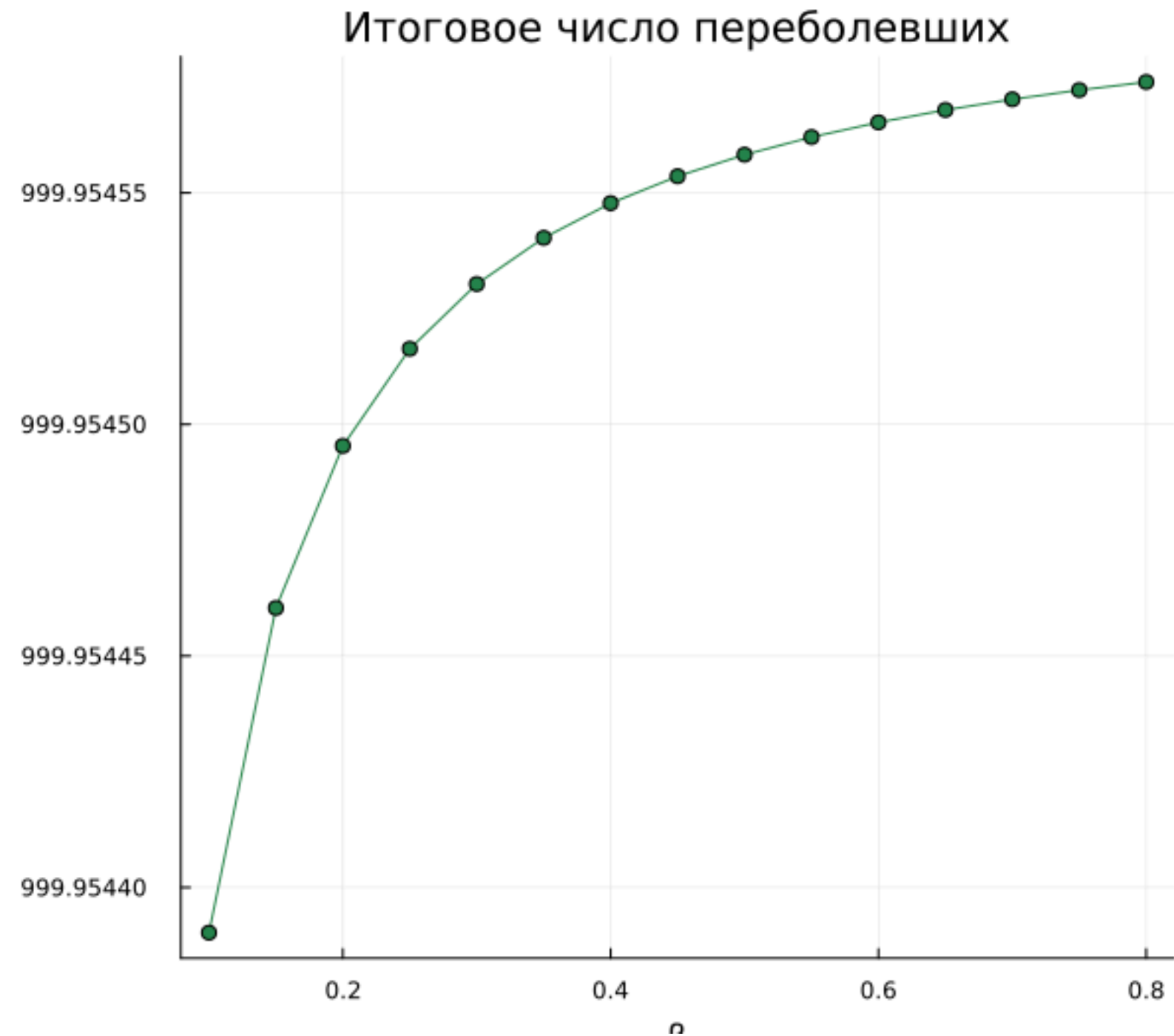
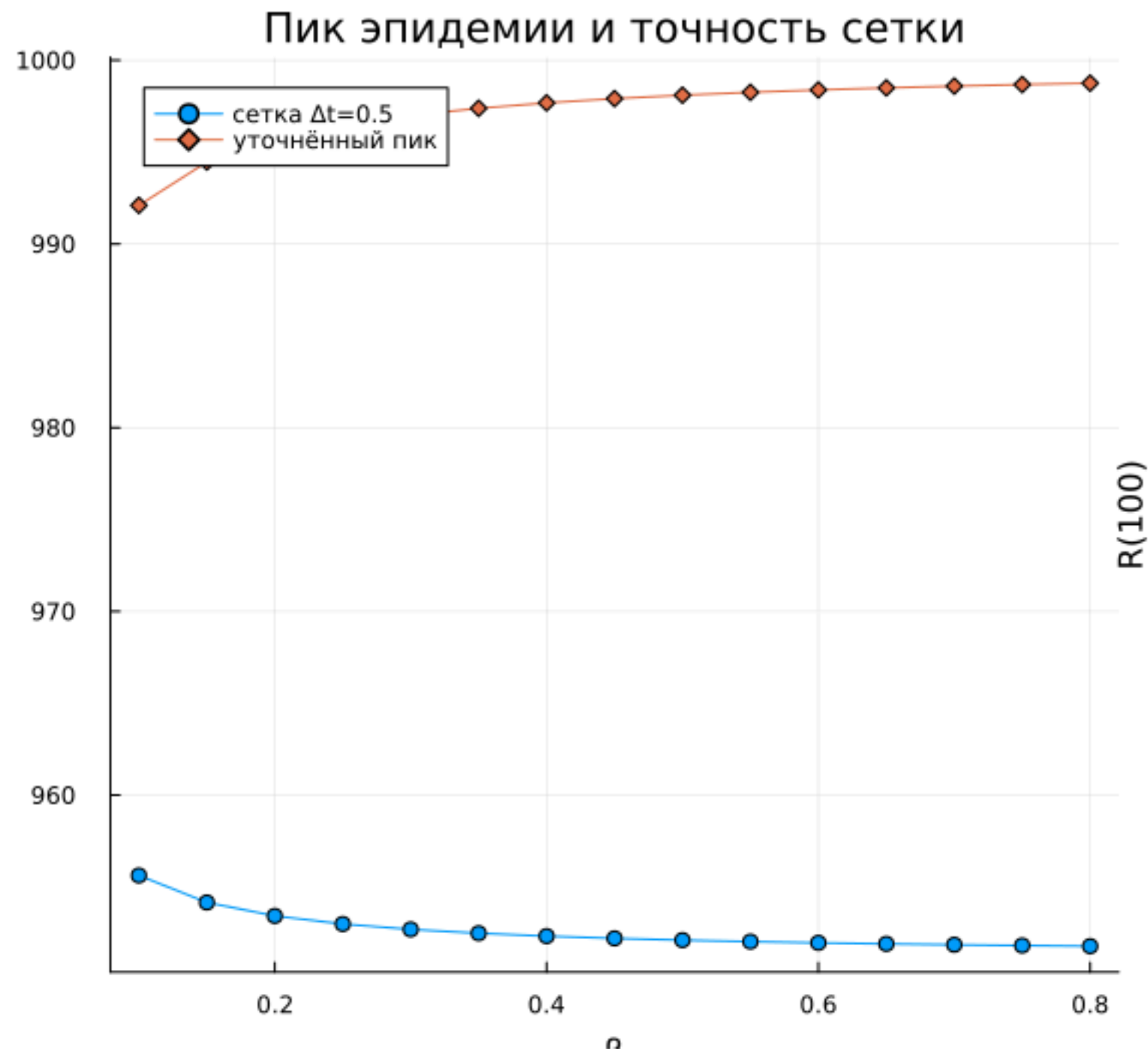
Сопоставление на моментах событий SSA

SSA — ступенчатый процесс. Значения сравниваются в фактические моменты событий, без линейного сглаживания дискретной маркировки.

Почему нужен уточнённый пик

- при βSI эпидемия развивается в самом начале интервала;
- сохранение ODE с шагом 0.5 пропускает истинный максимум;
- уточнение использует условие $S = \gamma/\beta$;
- значение проверяется аналитическим инвариантом SIR.

Сканирование β



15 обязательных значений β

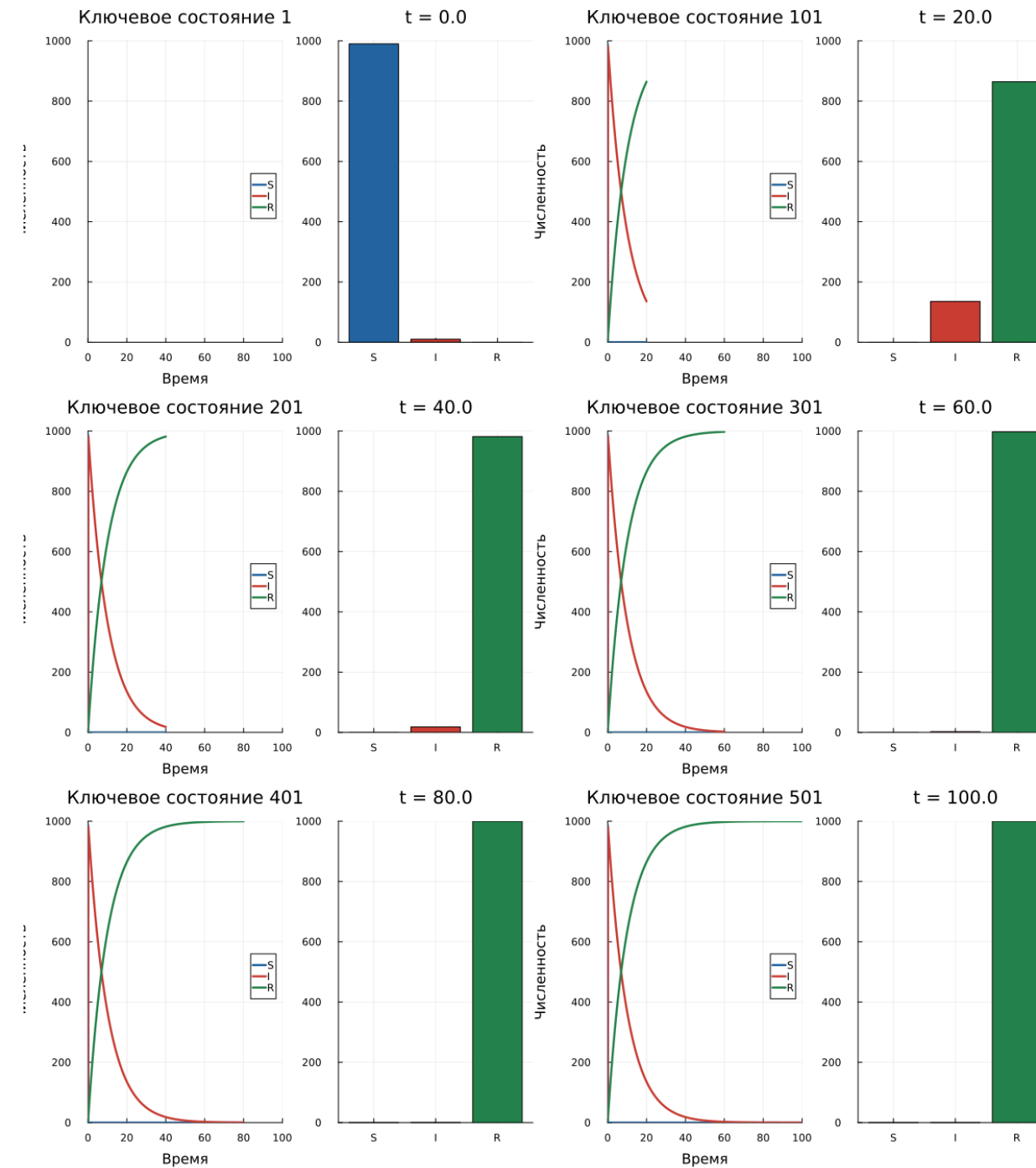
Порог роста

$$\dot{I}(0) > 0 \iff \beta S_0 - \gamma > 0$$

$$\beta > \frac{\gamma}{S_0} = \frac{0.1}{990} \approx 0.000101$$

Все значения обязательной сетки 0.1 : 0.05 : 0.8 выше этого порога.

Настоящая анимация

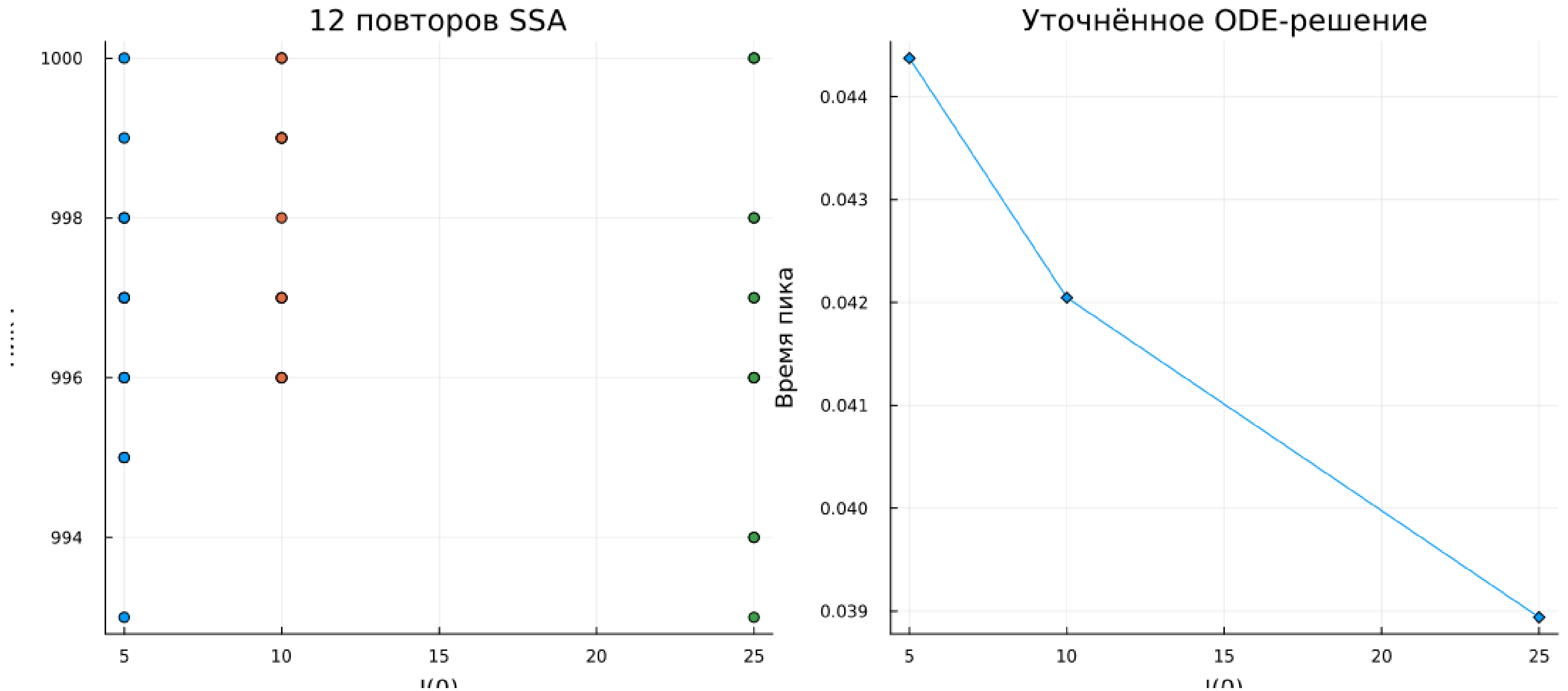


Шесть ключевых состояний

501 состояние: $t = 0 : 0.2 : 100$. Сценарий создаёт GIF, а не повторяет сканирование.

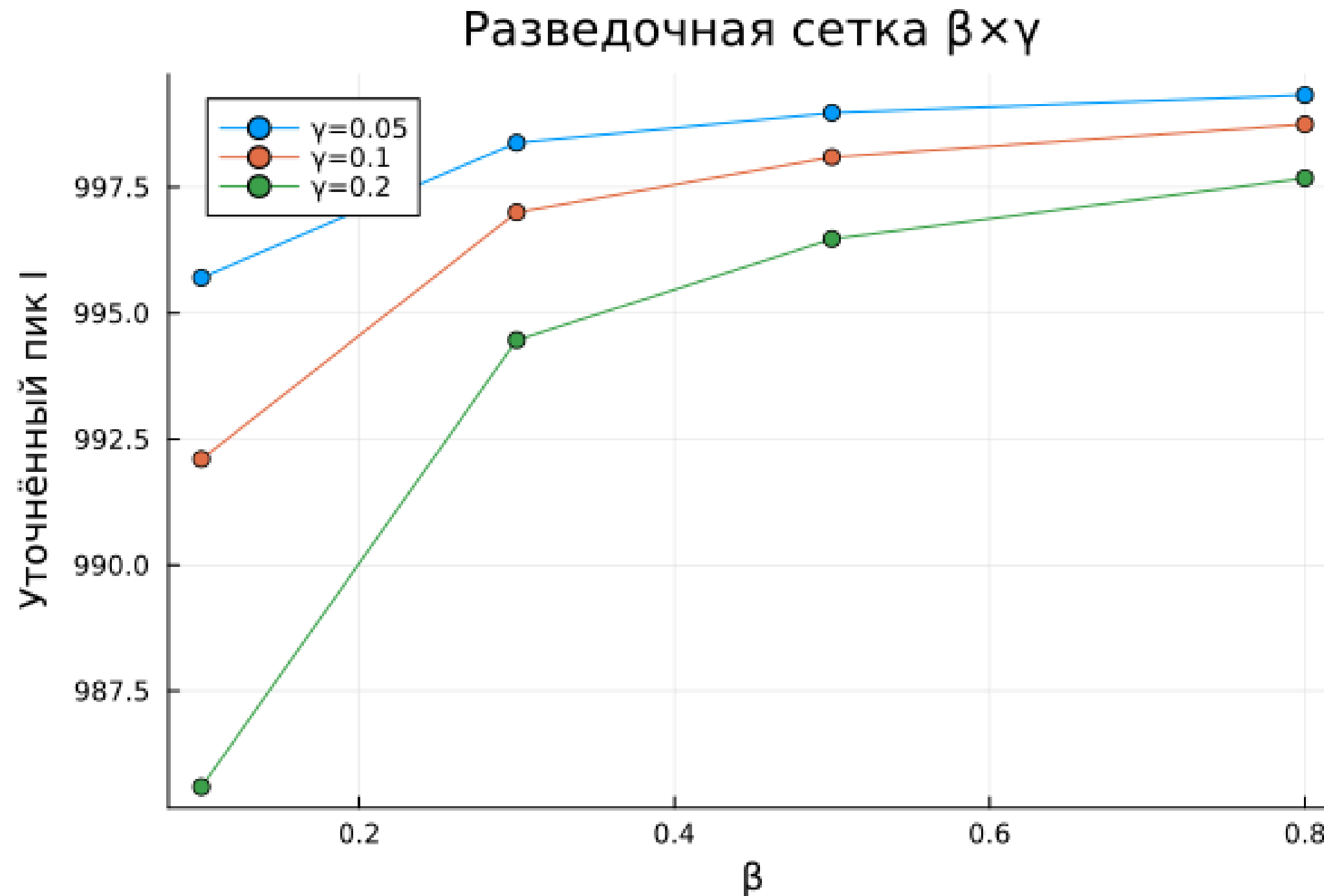
Дополнительные эксперименты

Начальное число инфицированных



Три начальных условия и 36 SSA-прогонов

Разведочная сетка $\beta \times \gamma$

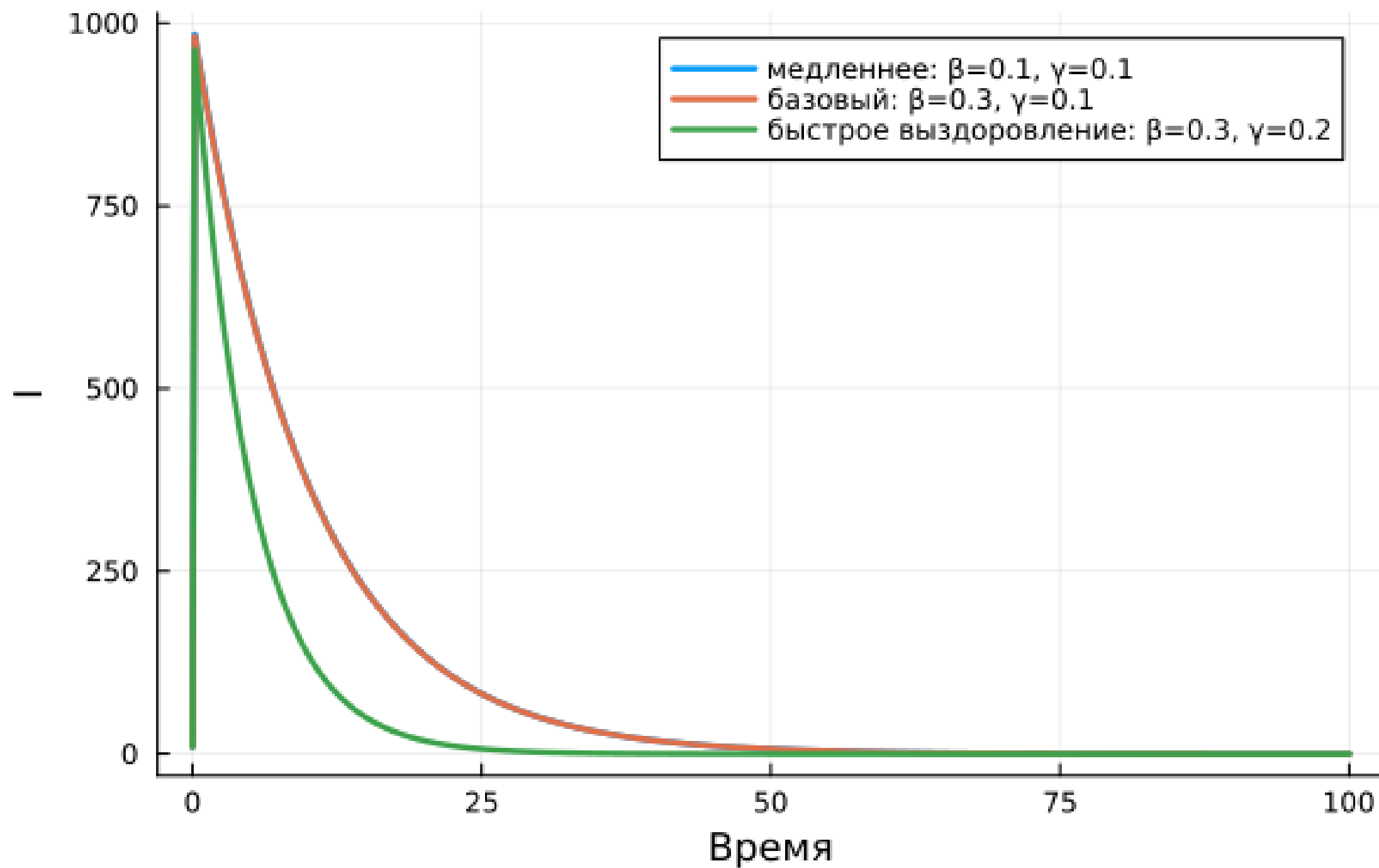


Полностью рассчитанные 12 комбинаций

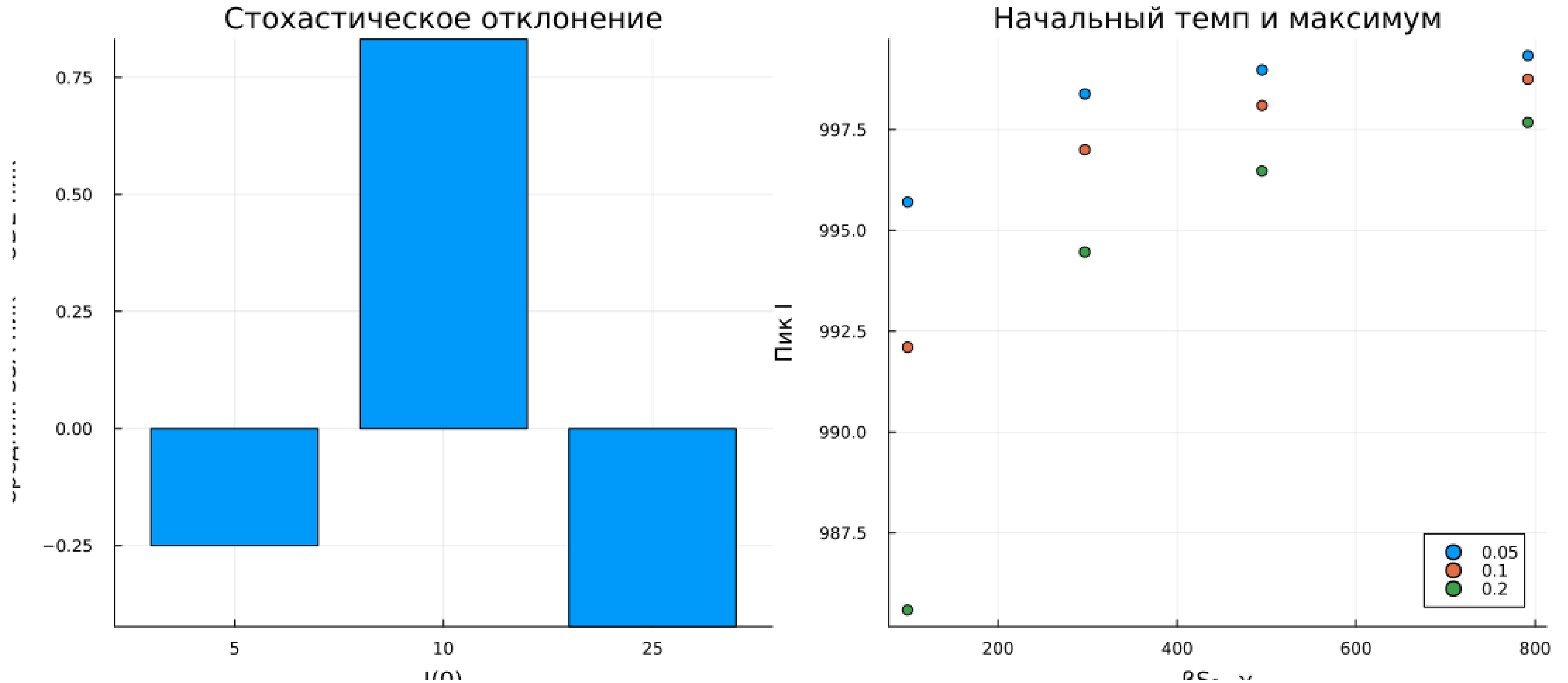
Сетка компактная и предварительная: без интерполяции и доверительных интервалов.

Три режима

Три режима модели SIR



Сводка параметрических результатов



Сравнение SSA и ODE

Проверка

Автоматические проверки

- матрица инцидентности равна ожидаемой;
- ODE содержит 201 точку при шаге 0.5;
- $S + I + R = 1000$ для ODE и SSA;
- уточнённый и аналитический пики совпадают;
- ранний пик находится до $t = 0.5$;
- правосторонняя выборка SSA сохраняет ступени.

Воспроизводимость

```
1 make run      # CSV, PNG и GIF
2 make notebooks # выполненные IPYNB
3 make docs     # автономные HTML
4 make test     # проверки модели
5 make verify   # контроль состава артефактов
```

Окружение закреплено [Project.toml](#) и [Manifest.toml](#).

Итоги

Получено

- настоящая сеть Петри с двумя переходами;
- ODE- и SSA-траектории при параметрах задания;
- сканирование 15 значений β и анимация на 501 состояние;
- параметрические эксперименты без скрытой подмены данных;
- литературные программы, ноутбуки, отчёт и презентация.

Вывод

Сеть Петри отделяет структуру SIR от способа вычисления. Один и тот же объект поддерживает непрерывное и событийное моделирование, а уточнённый анализ раннего пика предотвращает неверные выводы по редкой временной сетке.